



CETAM
CENTRO DE EDUCAÇÃO
TECNOLOGICA DO AMAZONAS



GOVERNO DO
AMAZONAS

Qualificação Profissional em Lógica de Programação Básica

GABARITO OFICIAL

Apostila de Lógica de Programação

com Portugol e VisuAlg

Resoluções comentadas de todos os exercícios propostos, capítulo a capítulo

Professora: Esp. Priscila Gonçalves

E-mail leylapristeacher@gmail.com

GitHb: https://github.com/priscilagon/Logica_Programacao_Basica

Manaus – Amazonas

Apresentação

Este gabarito apresenta uma proposta de resolução comentada para cada um dos exercícios propostos na Apostila de Lógica de Programação com Portugol e VisuAlg. Em exercícios de programação, os algoritmos apresentados são soluções de referência: existem outras formas corretas de resolver o mesmo problema, e pequenas variações de nomes de variáveis, mensagens ou organização do código não tornam a solução do aluno incorreta, desde que a lógica esteja certa e o resultado final seja o esperado.

Como usar este material

Ao corrigir os exercícios dos alunos, verifique principalmente:

- (1) se a lógica do algoritmo resolve corretamente o problema proposto;
- (2) se os tipos de dados e a sintaxe do VisuAlg foram respeitados;
- (3) se o algoritmo trata corretamente os casos de teste esperados.

Diferenças de estilo (nomes de variáveis, mensagens exibidas) não devem ser penalizadas.

Capítulo 1: Introdução à Lógica de Programação

Exercício 1: *Descreva, em uma sequência numerada de passos, como você faria para trocar uma lâmpada queimada.*

Resposta: Espera-se uma sequência finita e ordenada de passos, por exemplo:

- 1) Desligar o interruptor/disjuntor;
- 2) Pegar uma escada, se necessário;
- 3) Retirar a lâmpada queimada girando-a no sentido anti-horário;
- 4) Verificar a voltagem e o soquete da lâmpada nova;
- 5) Encaixar a lâmpada nova girando-a no sentido horário;
- 6) Ligar o interruptor e verificar se acendeu. O importante é avaliar se o aluno apresentou passos claros, em ordem lógica e sem ambiguidade.

Exercício 2: *Escreva o passo a passo lógico para atravessar uma rua com segurança.*

Resposta: Exemplo de sequência aceitável:

- 1) Ir até a faixa de pedestres;
- 2) Observar o sinal (se houver semáforo) ou olhar para os dois lados da rua;
- 3) Aguardar até que não haja veículos se aproximando;
- 4) Atravessar em linha reta, sem correr;
- 5) Continuar observando o trânsito até chegar ao outro lado. Avalie se o aluno incluiu a verificação de segurança antes de agir (uma espécie de "condição" antes de atravessar), o que já antecipa a ideia de estrutura condicional.

Exercício 3: *Pense em uma tarefa do seu cotidiano e identifique: quais são os dados de entrada, o processamento e a saída dessa tarefa?*

Resposta: Resposta pessoal/livre. Exemplo de referência: tarefa "calcular o troco de uma compra". Entrada: valor da compra e valor pago pelo cliente. Processamento: subtrair o valor da compra do valor pago. Saída: o valor do troco a ser devolvido. O que deve ser avaliado é se o aluno conseguiu separar corretamente os três elementos (entrada, processamento e saída) em uma tarefa real, e não a tarefa escolhida em si.

Exercício 4: *Explique, com suas palavras, a diferença entre lógica de programação e uma linguagem de programação.*

Resposta: Lógica de programação é a forma de pensar e organizar passos para resolver um problema, é independente de qualquer linguagem específica. Uma linguagem de programação (como Português, Python, Java ou C) é a ferramenta usada para expressar essa lógica em um formato que possa ser compreendido e executado por um computador. Ou seja, a lógica é o raciocínio; a linguagem é a forma de escrevê-lo. A mesma lógica pode ser implementada em diferentes linguagens.

Capítulo 2: Algoritmos e Fluxogramas

Exercício 1: *Descreva, em passos numerados, um algoritmo para verificar se um número digitado pelo usuário é par ou ímpar.*

Resposta:

- 1) Ler o número digitado;
- 2) Calcular o resto da divisão do número por 2;
- 3) Se o resto for igual a 0, o número é par;
- 4) Caso contrário, o número é ímpar;
- 5) Exibir o resultado. Verifique se o aluno usou a ideia de resto da divisão (mesmo sem ainda conhecer o operador mod) como critério de decisão.

Exercício 2: *Descreva um algoritmo para calcular a área de um retângulo, sabendo que o usuário informará a base e a altura.*

Resposta:

- 1) Ler o valor da base;
- 2) Ler o valor da altura;
- 3) Calcular a área multiplicando base por altura;
- 4) Exibir o valor da área calculada.

Exercício 3: *Explique, com suas palavras, o que representa o símbolo de losango em um fluxograma.*

Resposta: O losango representa uma decisão (condição) dentro do fluxograma. Ele indica um ponto em que o fluxo do algoritmo se divide em dois caminhos possíveis, de acordo com o resultado (verdadeiro ou falso) de uma condição testada por exemplo, "a nota é maior ou igual a 6?".

Exercício 4: *Escreva o algoritmo (em passos numerados) para determinar o maior entre três números informados pelo usuário.*

Resposta:

- 1) Ler o primeiro número (A);
- 2) Ler o segundo número (B);
- 3) Ler o terceiro número (C);
- 4) Se A for maior que B e A for maior que C, então A é o maior;
- 5) Senão, se B for maior que C, então B é o maior;
- 6) Senão, C é o maior;
- 7) Exibir o maior valor encontrado.

Capítulo 3: Conhecendo o VisuAlg e o Portugol

Exercício 1: Escreva, no VisuAlg, um algoritmo que exiba na tela seu nome completo e o curso que você está fazendo.

Resposta esperada (algoritmo comentado):

```
algoritmo "Apresentacao"
inicio
    escreval("Nome: Fulano de Tal Silva")
    escreval("Curso: Tecnico em Desenvolvimento de Sistemas")
finalgoritmo
```

Observação: o objetivo é verificar se o aluno usou corretamente a estrutura algoritmo...inicio...finalgoritmo e o comando escreval() com o próprio nome e curso, substituindo os dados de exemplo pelos seus.

Exercício 2: Explique com suas palavras a função das palavras reservadas algoritmo, var, inicio e finalgoritmo.

Resposta: algoritmo: define o nome do programa, sempre escrito entre aspas logo na primeira linha. var: inicia a seção onde são declaradas todas as variáveis que o programa vai utilizar, junto com seus tipos. inicio: marca o começo do bloco de comandos que efetivamente serão executados pelo programa. finalgoritmo: marca o final do programa, encerrando sua execução.

Exercício 3: Escreva um algoritmo que exiba, em linhas separadas, três frases diferentes usando o comando escreva().

Resposta esperada (algoritmo comentado):

```
algoritmo "Tres Frases"
inicio
    escreva("Estou aprendendo logica de programacao.")
    escreva("\n")
    escreva("O VisuAlg usa a linguagem Portugol.")
    escreva("\n")
    escreva("Praticar e a melhor forma de aprender.")
finalgoritmo
```

Observação: como o comando escreva() não pula linha automaticamente, é necessário usar \n ou alternar para escreval() para separar as frases em linhas diferentes. Ambas as abordagens devem ser aceitas.

Exercício 4: Pesquise e explique para que serve o comando escreval() no VisuAlg, e qual a diferença dele para o comando escreva().

Resposta: O comando escreval() funciona da mesma forma que o escreva(), exibindo textos e valores na tela, mas com uma diferença: ao final da exibição, ele insere automaticamente uma quebra de linha, fazendo com que o próximo comando de saída comece em uma nova linha. Já o escreva() mantém o cursor na mesma linha após a exibição, permitindo concatenar várias saídas seguidas na mesma linha, a menos que se use \n manualmente.

Capítulo 4: Variáveis, Constantes e Tipos de Dados

Exercício 1: Declare variáveis adequadas (com nome e tipo corretos) para armazenar: o nome de um produto, seu preço, a quantidade em estoque e se ele está em promoção.

Resposta esperada (algoritmo comentado):

```
var
  nome_produto: caractere
  preco: real
  quantidade_estoque: inteiro
  em_promocao: logico
```

Exercício 2: Explique a diferença entre uma variável do tipo inteiro e uma do tipo real, dando um exemplo de uso para cada uma.

Resposta: O tipo inteiro armazena apenas números inteiros, positivos ou negativos, sem casas decimais por exemplo, a quantidade de alunos em uma turma (idade: 20). O tipo real armazena números que podem ter casas decimais (ponto flutuante) por exemplo, o preço de um produto (preco: 19.90) ou a altura de uma pessoa (altura: 1.75).

Exercício 3: Escreva um algoritmo que declare uma constante para o valor do salário mínimo e uma variável para calcular o dobro desse valor.

Resposta esperada (algoritmo comentado):

```
algoritmo "Dobro do Salario Minimo"
const
  SALARIO_MINIMO = 1412.00
var
  dobro: real

inicio
  dobro <- SALARIO_MINIMO * 2
  escreval("O dobro do salario minimo e: ", dobro)
fimalgoritmo
```

Observação: o valor exato do salário mínimo pode variar conforme o ano/época em que o exercício for aplicado; o importante é o uso correto de const e do cálculo do dobro.

Exercício 4: Aponte o que está incorreto nos seguintes nomes de variáveis e reescreva-os corretamente: "2idade", "nome completo", "salário", "var".

Resposta: 2idade: inválido porque começa com número, pode ser corrigido para idade ou idade2. nome completo: inválido porque contém espaço, deve ser escrito como nome_completo. salário: inválido porque contém acento, deve ser escrito como salario. var: inválido porque é uma palavra reservada da linguagem, pode ser renomeado para variavel ou para um nome mais específico, como valor.

Capítulo 5: Operadores e Expressões

Exercício 1: Calcule manualmente o resultado das expressões: $2 + 3 * 4$; $(2 + 3) * 4$; $10 \text{ div } 3$; $10 \text{ mod } 3$.

Resposta: $2 + 3 * 4 = 2 + 12 = 14$ (a multiplicação tem precedência sobre a soma). $(2 + 3) * 4 = 5 * 4 = 20$ (o parêntese é resolvido primeiro). $10 \text{ div } 3 = 3$ (divisão inteira, despreza o resto). $10 \text{ mod } 3 = 1$ (resto da divisão de 10 por 3).

Exercício 2: Escreva a expressão lógica em Portugol que representa: "a pessoa pode votar se tiver 16 anos ou mais".

Resposta esperada (algoritmo comentado):

```
idade >= 16
```

Exercício 3: Escreva um algoritmo que leia dois números inteiros e exiba a soma, a subtração, o produto, a divisão real e o resto da divisão entre eles.

Resposta esperada (algoritmo comentado):

```
algoritmo "Operacoes Basicas"
var
    a, b: inteiro

inicio
    escreva("Digite o primeiro numero: ")
    leia(a)
    escreva("Digite o segundo numero: ")
    leia(b)

    escreval("Soma: ", a + b)
    escreval("Subtracao: ", a - b)
    escreval("Produto: ", a * b)
    escreval("Divisao real: ", a / b)
    escreval("Resto da divisao: ", a mod b)
fimalgoritmo
```

Exercício 4: Escreva a expressão lógica que verifica se um número é, ao mesmo tempo, maior que 0 e menor que 100.

Resposta esperada (algoritmo comentado):

```
(numero > 0) e (numero < 100)
```

Capítulo 6: Entrada e Saída de Dados

Exercício 1: Escreva um algoritmo que leia o nome e o preço de um produto e exiba uma mensagem informando os dois valores.

Resposta esperada (algoritmo comentado):

```
algoritmo "Dados do Produto"
var
    nome: caractere
    preco: real

inicio
    escreva("Digite o nome do produto: ")
    leia(nome)
    escreva("Digite o preço do produto: ")
    leia(preco)

    escreval("O produto ", nome, " custa R$ ", preco)
fimalgoritmo
```

Exercício 2: Escreva um algoritmo que leia dois números e exiba a soma entre eles, usando escreval().

Resposta esperada (algoritmo comentado):

```
algoritmo "Soma de Dois Numeros"
var
    a, b: real

inicio
    escreva("Digite o primeiro numero: ")
    leia(a)
    escreva("Digite o segundo numero: ")
    leia(b)

    escreval("A soma e: ", a + b)
fimalgoritmo
```

Exercício 3: Escreva um algoritmo que leia a base e a altura de um triângulo e calcule sua área ($\text{base} * \text{altura} / 2$).

Resposta esperada (algoritmo comentado):

```
algoritmo "Area do Triangulo"
var
    base, altura, area: real

inicio
    escreva("Digite a base do triangulo: ")
    leia(base)
    escreva("Digite a altura do triangulo: ")
    leia(altura)

    area <- (base * altura) / 2
    escreval("A area do triangulo e: ", area)
fimalgoritmo
```


Exercício 4: *Explique, com suas palavras, a diferença entre os comandos escreva() e escreval().*

Resposta: O comando escreva() exibe um texto ou valor na tela e mantém o cursor na mesma linha, permitindo que a próxima saída continue ao lado da anterior. Já o escreval() exibe o conteúdo e, em seguida, pula automaticamente para uma nova linha, fazendo com que a próxima saída apareça na linha seguinte.

Capítulo 7: Estruturas Condicionais

Exercício 1: Escreva um algoritmo que leia um número e informe se ele é par ou ímpar (dica: use o operador mod).

Resposta esperada (algoritmo comentado):

```
algoritmo "Par ou Impar"
var
    numero: inteiro

inicio
    escreva("Digite um numero: ")
    leia(numero)

    se (numero mod 2 = 0) entao
        escreval("O numero e par.")
    senao
        escreval("O numero e impar.")
    fimse
fimalgoritmo
```

Exercício 2: Escreva um algoritmo que leia três notas de um aluno, calcule a média e classifique o resultado em: "Aprovado" (média ≥ 7), "Recuperação" (média entre 5 e 6.9) ou "Reprovado" (média menor que 5).

Resposta esperada (algoritmo comentado):

```
algoritmo "Classificacao do Aluno"
var
    n1, n2, n3, media: real

inicio
    escreva("Digite a primeira nota: ")
    leia(n1)
    escreva("Digite a segunda nota: ")
    leia(n2)
    escreva("Digite a terceira nota: ")
    leia(n3)

    media <- (n1 + n2 + n3) / 3

    se (media >= 7) entao
        escreval("Aprovado")
    senao se (media >= 5) entao
        escreval("Recuperacao")
    senao
        escreval("Reprovado")
    fimse
fimalgoritmo
```

Exercício 3: Escreva um algoritmo, usando escolha...caso, que leia o número de um mês (1 a 12) e exiba o nome da estação do ano correspondente (considere o hemisfério sul).

Resposta esperada (algoritmo comentado):

```
algoritmo "Estacao do Ano"
var
```

```

mes: inteiro

inicio
  escreva("Digite o numero do mes (1 a 12): ")
  leia(mes)

  escolha mes
    caso 12, 1, 2
      escreval("Verão")
    caso 3, 4, 5
      escreval("Outono")
    caso 6, 7, 8
      escreval("Inverno")
    caso 9, 10, 11
      escreval("Primavera")
    outrocaso
      escreval("Mes invalido")
  fimescolha
fim algoritmo

```

Exercício 4: Escreva um algoritmo que leia três números e exiba o maior deles, utilizando estruturas condicionais.

Resposta esperada (algoritmo comentado):

```

algoritmo "Maior de Tres"
var
  a, b, c, maior: real

inicio
  escreva("Digite o primeiro numero: ")
  leia(a)
  escreva("Digite o segundo numero: ")
  leia(b)
  escreva("Digite o terceiro numero: ")
  leia(c)

  maior <- a
  se (b > maior) entao
    maior <- b
  fimse
  se (c > maior) entao
    maior <- c
  fimse

  escreval("O maior numero e: ", maior)
fim algoritmo

```

Capítulo 8 — Estruturas de Repetição

Exercício 1: Escreva um algoritmo que exiba, usando a estrutura para, todos os números pares entre 1 e 50.

Resposta esperada (algoritmo comentado):

```
algoritmo "Pares de 1 a 50"
var
    i: inteiro

inicio
    para i de 2 ate 50 passo 2 faca
        escreval(i)
    fimpara
fimalgoritmo
```

Solução alternativa (também correta): percorrer de 1 a 50 e testar se $i \bmod 2 = 0$ dentro do laço.

Exercício 2: Escreva um algoritmo que calcule o fatorial de um número digitado pelo usuário (dica: fatorial de 5 é $5*4*3*2*1$).

Resposta esperada (algoritmo comentado):

```
algoritmo "Fatorial"
var
    numero, i, fatorial: inteiro

inicio
    escreva("Digite um numero: ")
    leia(numero)

    fatorial <- 1
    para i de 1 ate numero faca
        fatorial <- fatorial * i
    fimpara

    escreval("O fatorial de ", numero, " e: ", fatorial)
fimalgoritmo
```

Exercício 3: Escreva um algoritmo que leia números digitados pelo usuário e pare quando for digitado um número negativo, exibindo ao final a soma e a média dos números positivos digitados.

Resposta esperada (algoritmo comentado):

```
algoritmo "Soma ate Numero Negativo"
var
    numero, soma, quantidade: inteiro
    media: real

inicio
    soma <- 0
    quantidade <- 0

    escreva("Digite um numero (negativo para parar): ")
    leia(numero)

    enquanto (numero >= 0) faca
        soma <- soma + numero
```

```

    quantidade <- quantidade + 1
    escreva("Digite um numero (negativo para parar): ")
    leia(numero)
fimenquanto

se (quantidade > 0) entao
    media <- soma / quantidade
    escreval("Soma: ", soma)
    escreval("Media: ", media)
senao
    escreval("Nenhum numero positivo foi digitado.")
fimse
fimalgoritmo

```

Exercício 4: Escreva um algoritmo, usando repita...ate, que simule um caixa eletrônico pedindo repetidamente uma senha de 4 dígitos até que o usuário acerte, permitindo no máximo 3 tentativas.

Resposta esperada (algoritmo comentado):

```

algoritmo "Caixa Eletronico"
var
    senha, tentativas: inteiro

inicio
    tentativas <- 0

    repita
        escreva("Digite a senha (1234): ")
        leia(senha)
        tentativas <- tentativas + 1
    ate (senha = 1234) ou (tentativas = 3)

    se (senha = 1234) entao
        escreval("Senha correta! Acesso liberado.")
    senao
        escreval("Numero maximo de tentativas excedido. Cartao bloqueado.")
    fimse
fimalgoritmo

```

Capítulo 9: Vetores

Exercício 1: Escreva um algoritmo que leia 10 números inteiros em um vetor e exiba o maior valor digitado.

Resposta esperada (algoritmo comentado):

```
algoritmo "Maior do Vetor"
var
    numeros: vetor[1..10] de inteiro
    i, maior: inteiro

inicio
    para i de 1 ate 10 faca
        escreva("Digite o numero ", i, ": ")
        leia(numeros[i])
    fimpara

    maior <- numeros[1]
    para i de 2 ate 10 faca
        se (numeros[i] > maior) entao
            maior <- numeros[i]
        fimse
    fimpara

    escreval("O maior valor digitado foi: ", maior)
fimalgoritmo
```

Exercício 2: Escreva um algoritmo que leia um vetor de 5 nomes (caractere) e depois exiba todos eles em ordem.

Resposta esperada (algoritmo comentado):

```
algoritmo "Vetor de Nomes"
var
    nomes: vetor[1..5] de caractere
    i: inteiro

inicio
    para i de 1 ate 5 faca
        escreva("Digite o nome ", i, ": ")
        leia(nomes[i])
    fimpara

    escreval("Nomes digitados:")
    para i de 1 ate 5 faca
        escreval(nomes[i])
    fimpara
fimalgoritmo
```

Exercício 3: Escreva um algoritmo que leia 5 números em um vetor e informe quantos deles são pares.

Resposta esperada (algoritmo comentado):

```
algoritmo "Contar Pares no Vetor"
var
    numeros: vetor[1..5] de inteiro
    i, contador: inteiro
```

```

inicio
  contador <- 0
  para i de 1 ate 5 faca
    escreva("Digite o numero ", i, ": ")
    leia(numeros[i])
    se (numeros[i] mod 2 = 0) entao
      contador <- contador + 1
    fimse
  fimpara

  escreval("Quantidade de numeros pares: ", contador)
finalgoritmo

```

Exercício 4: Escreva um algoritmo que leia 8 números em um vetor e calcule a soma e a média dos valores armazenados.

Resposta esperada (algoritmo comentado):

```

algoritmo "Soma e Media do Vetor"
var
  numeros: vetor[1..8] de real
  i: inteiro
  soma, media: real

inicio
  soma <- 0
  para i de 1 ate 8 faca
    escreva("Digite o numero ", i, ": ")
    leia(numeros[i])
    soma <- soma + numeros[i]
  fimpara

  media <- soma / 8
  escreval("Soma: ", soma)
  escreval("Media: ", media)
finalgoritmo

```

Capítulo 10: Matrizes

Exercício 1: Escreva um algoritmo que leia uma matriz 3x3 de números inteiros e exiba a soma de cada linha.

Resposta esperada (algoritmo comentado):

```
algoritmo "Soma por Linha"
var
    matriz: vetor[1..3, 1..3] de inteiro
    linha, coluna, soma_linha: inteiro

inicio
    para linha de 1 ate 3 faca
        para coluna de 1 ate 3 faca
            escreva("Digite o valor [", linha, ",", coluna, "]: ")
            leia(matriz[linha, coluna])
        fimpara
    fimpara

    para linha de 1 ate 3 faca
        soma_linha <- 0
        para coluna de 1 ate 3 faca
            soma_linha <- soma_linha + matriz[linha, coluna]
        fimpara
        escreval("Soma da linha ", linha, ": ", soma_linha)
    fimpara
fimalgoritmo
```

Exercício 2: Escreva um algoritmo que leia uma matriz 2x4 e exiba o maior valor encontrado, informando também em que linha e coluna ele está.

Resposta esperada (algoritmo comentado):

```
algoritmo "Maior da Matriz"
var
    matriz: vetor[1..2, 1..4] de inteiro
    linha, coluna, maior, linha_maior, coluna_maior: inteiro

inicio
    para linha de 1 ate 2 faca
        para coluna de 1 ate 4 faca
            escreva("Digite o valor [", linha, ",", coluna, "]: ")
            leia(matriz[linha, coluna])
        fimpara
    fimpara

    maior <- matriz[1, 1]
    linha_maior <- 1
    coluna_maior <- 1

    para linha de 1 ate 2 faca
        para coluna de 1 ate 4 faca
            se (matriz[linha, coluna] > maior) entao
                maior <- matriz[linha, coluna]
                linha_maior <- linha
                coluna_maior <- coluna
            fimse
        fimpara
    fimpara
```



```
fimpara

escreval("Maior valor: ", maior)
escreval("Posicao: linha ", linha_maior, ", coluna ", coluna_maior)
fimalgoritmo
```

Exercício 3: Escreva um algoritmo que leia uma matriz 3x3 e calcule a soma dos elementos da diagonal principal (posições onde linha = coluna).

Resposta esperada (algoritmo comentado):

```
algoritmo "Diagonal Principal"
var
    matriz: vetor[1..3, 1..3] de inteiro
    linha, coluna, soma: inteiro

inicio
    para linha de 1 ate 3 faca
        para coluna de 1 ate 3 faca
            escreva("Digite o valor [", linha, ",", coluna, "]: ")
            leia(matriz[linha, coluna])
        fimpara
    fimpara

    soma <- 0
    para linha de 1 ate 3 faca
        soma <- soma + matriz[linha, linha]
    fimpara

    escreval("Soma da diagonal principal: ", soma)
fimalgoritmo
```

Observação: como a diagonal principal ocorre sempre quando linha = coluna, basta um único laço para percorrê-la, usando matriz[linha, linha].

Exercício 4: Escreva um algoritmo que leia uma matriz 4x2 representando o nome de um produto e sua quantidade em estoque, e exiba somente os produtos com quantidade menor que 10.

Resposta esperada (algoritmo comentado):

```
algoritmo "Produtos com Estoque Baixo"
var
    produtos: vetor[1..4, 1..2] de caractere
    i: inteiro

inicio
    para i de 1 ate 4 faca
        escreva("Nome do produto ", i, ": ")
        leia(produtos[i, 1])
        escreva("Quantidade em estoque: ")
        leia(produtos[i, 2])
    fimpara

    escreval("Produtos com estoque baixo (menor que 10):")
    para i de 1 ate 4 faca
        se (produtos[i, 2] < "10") entao
            escreval(produtos[i, 1])
        fimse
    fimpara
```

```
fimpara
fimalgoritmo
```

Observação pedagógica

Como uma matriz é homogênea (todas as posições do mesmo tipo), armazenar nome (texto) e quantidade (número) juntos exige que ambos sejam do tipo caractere, o que dificulta a comparação numérica. A solução mais correta e recomendada é usar dois vetores paralelos, um vetor de caractere para os nomes e um vetor de inteiro para as quantidades, em vez de uma única matriz. Aceite também essa solução alternativa, mostrada a seguir, como resposta preferencial.

```
algoritmo "Produtos com Estoque Baixo (vetores paralelos)"
var
    nomes: vetor[1..4] de caractere
    quantidades: vetor[1..4] de inteiro
    i: inteiro

inicio
    para i de 1 ate 4 faca
        escreva("Nome do produto ", i, ": ")
        leia(nomes[i])
        escreva("Quantidade em estoque: ")
        leia(quantidades[i])
    fimpara

    escreval("Produtos com estoque baixo (menor que 10):")
    para i de 1 ate 4 faca
        se (quantidades[i] < 10) entao
            escreval(nomes[i])
        fimse
    fimpara
fimalgoritmo
```

Capítulo 11: Arquivos

Exercício 1: Escreva um algoritmo que grave em um arquivo os nomes e as idades de 5 pessoas digitadas pelo usuário.

Resposta esperada (algoritmo comentado):

```
algoritmo "Gravar Pessoas"
var
    arq: arquivo
    nome: caractere
    idade, i: inteiro

inicio
    atribuir arq arquivo "pessoas.txt"
    reescreva(arq)

    para i de 1 ate 5 faca
        escreva("Nome da pessoa ", i, ": ")
        leia(nome)
        escreva("Idade: ")
        leia(idade)
        escreva(arq, nome, ";", idade)
    fimpara

    fecha(arq)
    escreval("Dados gravados com sucesso!")
fimalgoritmo
```

Exercício 2: Escreva um algoritmo que leia o arquivo criado no exercício anterior e exiba na tela todas as informações gravadas.

Resposta esperada (algoritmo comentado):

```
algoritmo "Ler Pessoas"
var
    arq: arquivo
    linha: caractere

inicio
    atribuir arq arquivo "pessoas.txt"
    abreva(arq)

    enquanto nao fimarq(arq) faca
        leia(arq, linha)
        escreval(linha)
    fimenquanto

    fecha(arq)
fimalgoritmo
```

Exercício 3: Explique, com suas palavras, a diferença entre os comandos `reescreva()` e `abreva()`.

Resposta: O comando `reescreva()` abre um arquivo para escrita: se o arquivo ainda não existir, ele é criado; se já existir, todo o seu conteúdo anterior é apagado antes de começar a gravar os novos dados. Já o comando `abreva()` abre um arquivo já existente apenas para leitura, permitindo ler os dados que já estão gravados nele, sem apagar ou alterar o conteúdo.

Exercício 4: Escreva um algoritmo que grave em um arquivo os produtos de uma lista de compras (nome e preço) e, em seguida, leia o arquivo e exiba o valor total da lista.

Resposta esperada (algoritmo comentado):

```
algoritmo "Lista de Compras"
var
    arq: arquivo
    nome: caractere
    preco, total: real
    continuar: caractere

inicio
    atribuir arq arquivo "compras.txt"
    reescreva(arq)

    repita
        escreva("Nome do produto: ")
        leia(nome)
        escreva("Preço do produto: ")
        leia(preco)
        escreva(arq, nome, ";", preco)

        escreva("Deseja adicionar outro produto? (S/N): ")
        leia(continuar)
    ate (continuar = "N") ou (continuar = "n")
    fecha(arq)

    // Leitura e soma dos preços gravados
    total <- 0
    abreva(arq)
    enquanto nao fimarq(arq) faca
        leia(arq, nome, preco)
        total <- total + preco
    fimenquanto
    fecha(arq)

    escreval("Valor total da lista de compras: R$ ", total)
fimalgoritmo
```

Capítulo 12: Projeto Prático Final

Os desafios extras deste capítulo pedem alterações no sistema de cadastro de produtos apresentado na apostila. A seguir, são descritas as ideias centrais para cada solução; o aluno pode implementá-las diretamente sobre o código-base fornecido no Capítulo 12 da apostila.

Exercício 1: Adicione ao sistema uma opção para buscar um produto pelo nome e exibir seu preço.

Resposta esperada (algoritmo comentado):

```

caso 6
  escreva("Digite o nome do produto a buscar: ")
  leia(busca)
  encontrado <- falso
  para i de 1 ate quantidade faca
    se (nomes[i] = busca) entao
      escreval("Produto encontrado! Preço: R$ ", precos[i])
      encontrado <- verdadeiro
  fimse
fimpara
se (nao encontrado) entao
  escreval("Produto nao encontrado.")
fimse

```

É necessário declarar as variáveis busca: caractere e encontrado: logico, além de acrescentar a nova opção no menu.

Exercício 2: Adicione uma opção para calcular e exibir o preço médio dos produtos cadastrados.

Resposta esperada (algoritmo comentado):

```

caso 7
  se (quantidade > 0) entao
    total <- 0
    para i de 1 ate quantidade faca
      total <- total + precos[i]
    fimpara
    escreval("Preco medio: R$ ", total / quantidade)
  senao
    escreval("Nenhum produto cadastrado.")
  fimse

```

Exercício 3: Modifique o sistema para não permitir o cadastro de produtos com preço negativo, exibindo uma mensagem de erro nesse caso.

Resposta esperada (algoritmo comentado):

```

caso 1
  se (quantidade < MAX) entao
    quantidade <- quantidade + 1
    escreva("Nome do produto: ")
    leia(nomes[quantidade])
    escreva("Preco do produto: ")
    leia(precos[quantidade])
    se (precos[quantidade] < 0) entao
      escreval("Preco invalido! Cadastro cancelado.")
      quantidade <- quantidade - 1
    fimse
  fimse

```

```
senao
    escreval("Estoque cheio!")
fimse
```

Exercício 4: Adicione uma opção que exiba o produto mais caro e o mais barato cadastrados.

Resposta esperada (algoritmo comentado):

```
caso 8
    se (quantidade > 0) entao
        indice_maior <- 1
        indice_menor <- 1
        para i de 2 ate quantidade faca
            se (precos[i] > precos[indice_maior]) entao
                indice_maior <- i
            fimse
            se (precos[i] < precos[indice_menor]) entao
                indice_menor <- i
            fimse
        fimpara
        escreval("Mais caro: ", nomes[indice_maior], " - R$ ",
precos[indice_maior])
        escreval("Mais barato: ", nomes[indice_menor], " - R$ ",
precos[indice_menor])
    senao
        escreval("Nenhum produto cadastrado.")
    fimse
```

Exercício 5: Adapte o sistema para usar uma matriz de estoque por filial (por exemplo, uma matriz 3x5, representando 3 filiais com 5 produtos cada) em vez de um único vetor.

Resposta esperada (algoritmo comentado):

```
algoritmo "Estoque por Filial"
const
    FILIAIS = 3
    PRODUTOS_POR_FILIAL = 5
var
    estoque: vetor[1..FILIAIS, 1..PRODUTOS_POR_FILIAL] de inteiro
    filial, produto, total: inteiro

inicio
    para filial de 1 ate FILIAIS faca
        para produto de 1 ate PRODUTOS_POR_FILIAL faca
            escreva("Filial ", filial, " - quantidade do produto ", produto, ":
")
                leia(estoque[filial, produto])
            fimpara
        fimpara

        para filial de 1 ate FILIAIS faca
            total <- 0
            para produto de 1 ate PRODUTOS_POR_FILIAL faca
                total <- total + estoque[filial, produto]
            fimpara
            escreval("Total de itens na filial ", filial, ": ", total)
        fimpara
    fimalgoritmo
```

Observação: o objetivo principal deste desafio é verificar se o aluno compreendeu a lógica de índices duplos (linha = filial, coluna = produto) e conseguiu adaptar a estrutura de repetição simples do vetor para uma estrutura de repetição dupla (aninhada) da matriz. Nomes de produtos por filial podem ser acrescentados com vetores paralelos de caractere, seguindo o mesmo raciocínio do Capítulo 10.